

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 01 · LECTURE NOTES

What Is Hard About Language

Four properties, and the whole course as a response.

MODULE OUTCOMES

- Explain discreteness and what it costs.
- Explain compositionality and ambiguity.
- Explain why the structure is not given.
- Map each course technique to the property it addresses.
- Explain why some tasks are harder than their framing suggests.

1 Four properties

Property	Why it makes language hard	The response, and where
Discrete	There is no natural distance between words, and the vocabulary is open-ended.	Learned embeddings and subword tokenisation (Modules 02 and 04).
Compositional	Meaning depends on how the words combine, not on which words are present.	Sequence models and attention (Modules 05 and 06).
Ambiguous	At every level simultaneously, and resolved by context (§2).	Contextual representations — a word's vector depending on its sentence (Module 07).
Unstructured	The syntax, semantics, and world knowledge are latent rather than labelled (§3).	Self-supervised pretraining on raw text (Module 07).

This table is the course's outline. Every technique in the remaining twelve modules is an answer to one of these four rows, and checking which one as you go is the most useful orientation device available — it converts a sequence of architectures into a sequence of responses to stated problems, which is both easier to remember and more transferable.

DISCRETENESS IS THE ONE THAT SHAPES EVERYTHING DOWNSTREAM

Pixels have a natural metric — two similar colours are numerically close, which is exactly why a convolution over them means something (CSCE 636 Module 05's whole premise). **Words do not have this property at all.**

'Cat' and 'dog' are no closer than 'cat' and 'thermodynamics' under any encoding you get for free — and a **one-hot vector makes every pair of words exactly equidistant**, which discards precisely the structure you needed and leaves the model to rediscover all of it from data.

And the vocabulary is open. New words, names, misspellings, and compounds appear constantly, so a fixed vocabulary guarantees unknown tokens at inference time — which is the problem Module 02's subword tokenisation exists to solve, and which the classical approach handled by mapping everything unseen to a single uninformative symbol.

So the first problem in natural language processing is representation rather than modelling — and **nearly all of the field's progress has come from better representations rather than better classifiers**, which is the single most useful generalisation available about its history and is worth holding onto through the next twelve modules.

2 Ambiguity

```

LEXICAL      "bank" -- financial or riverside
MORPHOLOGICAL
              "unlockable" -- cannot be locked, or able
              to be unlocked
SYNTACTIC    "I saw the man with the telescope"
              -- who had the telescope?
SCOPE        "every student read a book" -- one book
              between them, or one each?
REFERENCE    "the trophy did not fit in the suitcase
              because it was too large" -- which one?
PRAGMATIC    "can you pass the salt?" -- not actually a
              question about your ability
  
```

AND THEY INTERACT. Resolving the syntax may require knowing the word sense; knowing the sense may require the syntax.

WHICH IS WHY the classical pipeline architecture (tokenise, tag, parse, then interpret) loses to joint models: each stage needs information that only the later stages have.

The interaction is the real difficulty, rather than any individual level — **which is the argument against the classical pipeline and for end-to-end models**. A pipeline commits to a tokenisation, then to a tagging, then to a parse, and **each commitment is made without the information that would have resolved it**, with the errors compounding downstream. **This critique motivates everything from Module 05 onward**,

and it is worth knowing that the pipeline was not abandoned because it was inelegant but because it was measurably worse.

AMBIGUITY IS RESOLVED BY CONTEXT, WHICH IS WHY CONTEXT LENGTH MATTERS

Nearly every ambiguity in §2 is resolved by surrounding text — sometimes a word away, sometimes a paragraph away, and sometimes by knowledge that is not in the text at all (the trophy example requires knowing how physical containment works).

Which sets the architectural requirement directly: a representation of a word must depend on its context — and **a single fixed vector per word type cannot possibly disambiguate 'bank'**, because it has to represent both senses at once (Module 04 §4's central limitation).

And it explains why context length is a headline number for these systems. A model that can only attend over five words cannot resolve a reference twenty words back, however good its parameters are — the information is simply not available to it.

So Module 04's static embeddings, Module 05's recurrence, and Module 06's attention are three successive answers to the same requirement, each handling more context more flexibly than the last — **which is why this course is ordered historically rather than by elegance.**

3 The missing structure

LANGUAGE HAS RICH STRUCTURE AND ALMOST NO LABELS

Syntax, semantics, discourse relations, and world knowledge are all present in text, and none of them are annotated — and hand-annotating them is expensive, slow, requires trained linguists, and produces datasets of thousands or at best hundreds of thousands of examples rather than the billions that modern methods consume.

But the text itself is abundant and free. Which makes self-supervision the obvious move: construct a prediction task from the text alone, and arrange matters so that the structure must be learned in order to solve it.

'Predict the next word' requires syntax (to know what part of speech comes next), semantics (to know which words make sense), and a very great deal of world knowledge (to complete 'the capital of France is') — so a model trained only on that objective acquires representations useful for tasks it never saw (Module 07).

Which is the single most consequential idea in modern natural language processing — and it follows directly from this property of the data rather than from any architectural insight, which is worth noticing: the transformer made it practical at scale, and the idea itself is a response to the absence of labels.

4 Tasks that are harder than their framing

- **Sentiment classification sounds easy** and includes sarcasm, negation at a distance ('not the worst film I have seen'), mixed sentiment about different aspects, and comparative statements — **each of which requires composition rather than keyword counting**, which is why the bag-of-words baseline plateaus where it does.
- **Named entity recognition** requires knowing that a particular sequence of entirely ordinary words is the title of something — **which is world knowledge rather than a surface pattern**, and is why the task benefits so much from pretraining.
- **Coreference resolution** requires the physical and social reasoning of the trophy example — **and that reasoning is not in the sentence**, which is what makes the Winograd-style examples a genuine test rather than a trick.
- **Translation** requires making choices the source language did not make: grammatical gender, levels of formality, definiteness, and number distinctions that are simply unspecified in the input — **so the model must infer or default, and both introduce error and bias** (Module 12 §2).
- **And summarisation has no single correct answer**, which **makes it an evaluation problem at least as much as a modelling one** (Module 10 §3). **The pattern across all five is that each task needs information not present in its input** — which is exactly why pretraining on everything helps more than designing a task-specific architecture, and is the empirical result that reorganised the field.

HOW TO READ THIS COURSE

Modules 02 to 04 address representation — tokenisation, counting-based models, and learned dense vectors, which is §1's discreteness row and the foundation for everything else.

Modules 05 to 07 address composition and context — recurrence, attention, the transformer, and pretraining, which covers the compositional, ambiguous, and unstructured rows together.

Modules 08 to 11 are practice — adaptation under real constraints, decoding, evaluation, and retrieval augmentation, which is where an actual system gets built and where Project 2 lives.

And Modules 12 and 13 are about failure and about honest claims — **which are not an appendix**, because **a fluent wrong answer is the characteristic failure of everything built in this course**, and fluency is precisely what these models are trained to produce.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jurafsky & Martin, chapters 1 and 2 (free)	§1 and §2 — the properties and the ambiguity levels, with far more examples.
Bender & Koller — Climbing towards NLU (free)	§3 and §4 argued sharply — what learning from form alone can and cannot provide. Read it again before Module 13.
Levesque et al. — The Winograd Schema Challenge (free)	§2's reference ambiguity as a designed test — and the reasoning about why it resists surface statistics.
Stanford CS224N, lecture 1 (free)	§1 as a lecture , with the representation argument made carefully.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a sentence ambiguous at three of the six levels** simultaneously.
- 2 Find a real ambiguity** whose resolution requires knowledge outside the text.
- 3 Compute the cosine similarity** of one-hot vectors for 'cat', 'dog', and 'thermodynamics'.
- 4 Explain what that result costs** a downstream classifier.
- 5 Estimate the unknown-token rate** of a 20,000-word vocabulary on held-out text.
- 6 Run a classical pipeline** on five sentences and find one compounding error.
- 7 Construct a sentiment example** that keyword counting gets wrong.

- 8 **Find a translation choice** the source language leaves unspecified.
- 9 **Write three summaries of one paragraph** and argue all three are acceptable.
- 10 **Map each of the four properties** to a technique you expect to meet, and check your predictions at Module 07.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four properties and the response to each.
- 2 Why does discreteness shape everything downstream?
- 3 What does a one-hot encoding discard?
- 4 Why does an open vocabulary guarantee unknown tokens?
- 5 Name the six ambiguity levels with an example of each.
- 6 Why do the levels interacting defeat a pipeline?
- 7 Why must a word's representation depend on its context?
- 8 Why was self-supervision inevitable here?
- 9 What does 'predict the next word' require the model to learn?
- 10 Give five tasks whose framing understates them, and the shared pattern.